

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 8

Professor Mario Pego

mario.pegos@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Elaborar a descrição URDF de um robô móvel

O que vamos ver

1

O que é URDF e para que serve

A descrição de hardware que o Gazebo vai usar na Etapa 9

2

Anatomia de link e joint

visual, collision, inertial — e os tipos de junta

3

Calculando o inertial

Fórmulas simples para caixa, cilindro e esfera

4

robo_hospitalar.urdf

O arquivo completo do AT Exercício 3

Onde estamos no AT

Etapa 7 fechou o lado 'software' do robô hospitalar

- `monitor_transporte.py` (Etapa 7): tópicos, serviço e parâmetro — o comportamento do robô
- Até agora, o robô só existe como código — nenhum arquivo descreve sua forma física
- Hoje descrevemos o **HARDWARE**: dimensões, formato e como as peças se conectam
- Essa descrição (URDF) é o que a Etapa 9 vai usar para colocar o robô dentro do Gazebo

URDF: Unified Robot Description Format

Um arquivo XML que descreve a forma física de um robô

- URDF é um formato XML padrão do ROS para descrever links (peças rígidas) e joints (juntas que os conectam)
- Não tem nada de 'código' nele — é dados estruturados, como o package.xml, mas descrevendo geometria em vez de dependências
- O robot_state_publisher lê um URDF e publica as transformações (TF) entre as peças do robô
- O Gazebo (Etapa 9) lê o MESMO URDF para saber que formas físicas simular — por isso escrever certo agora evita retrabalho depois

A estrutura mínima de um URDF

Um <robot> contém vários <link> e <joint>

```
<robot name="meu_robô">

  <link name="...">...</link>
  <link name="...">...</link>

  <joint name="..." type="...">
    <parent link="..."/>
    <child link="..."/>
  </joint>

</robot>
```

link vs joint

Um link é uma peça rígida (não se deforma).

Um joint conecta exatamente dois links (parent e child) e define como um se move em relação ao outro.

Anatomia de um <link>

Três blocos, cada um com um papel diferente

```
<link name="exemplo">
  <visual>
    <geometry>...</geometry>
  </visual>
  <collision>
    <geometry>...</geometry>
  </collision>
  <inertial>
    <mass value="..." />
    <inertia .../>
  </inertial>
</link>
```

Por que três, não um

visual é o que se DESENHA (RViz/Gazebo).
collision é a forma usada para detectar colisões (pode ser mais simples que a visual, para economizar processamento). inertial é a massa e distribuição de massa, usada pela física do Gazebo. O AT exige os três em todo link.

As três formas geométricas mais comuns

Usadas dentro de <visual> e <collision>

Geometria	Parâmetros	Uso típico
<code><box size="x y z"/></code>	comprimento, largura, altura	chassi, corpos retangulares
<code><cylinder radius="r" length="l"/></code>	raio, comprimento	rodas, eixos
<code><sphere radius="r"/></code>	raio	rodas de apoio (caster), juntas esféricas

<origin>: posição e rotação relativas

xyz em metros, rpy em radianos

```
<origin xyz="0.2 0 -0.075"  
        rpy="1.5708 0 0"/>  
  
<!-- xyz: deslocamento em metros  
      rpy: rotação (roll,pitch,yaw) em radianos  
      1.5708 rad = 90° -->
```

Cuidado com a unidade

URDF usa RADIADOS, não graus. $90^\circ = 1.5708 \text{ rad}$, $180^\circ = 3.1416 \text{ rad}$. É o erro mais comum de quem vem de outras ferramentas de modelagem 3D.

Tipos de <joint>

O AT usa dois: continuous (rodas) e fixed (roda de apoio)

Tipo	Movimento permitido	Uso no robo_hospitalar
fixed	Nenhum — solda os dois links	roda_apoio (não gira, só sustenta)
continuous	Rotação livre, sem limite, em torno de <axis>	roda_esq e roda_dir (giram para sempre)
revolute	Rotação limitada (com <limit>)	um braço robótico com fim de curso, por exemplo
prismatic	Deslizamento linear, com <limit>	um pistão, uma gaveta

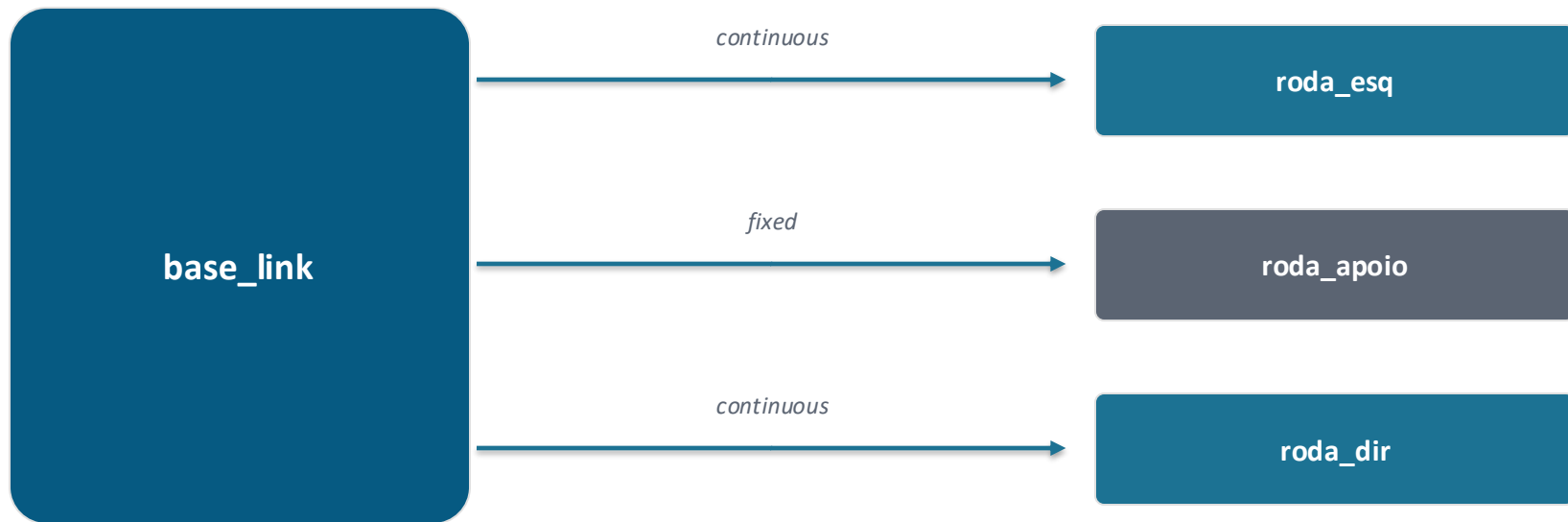
Calculando o <inertial>: fórmulas prontas

m = massa; para caixa, x/y/z = dimensões; para cilindro/esfera, r = raio

Forma	I_{xx}	I_{yy}	I_{zz}
Caixa (x,y,z)	$m(y^2+z^2)/12$	$m(x^2+z^2)/12$	$m(x^2+y^2)/12$
Cilindro (r,h), eixo=Z	$m(3r^2+h^2)/12$	$m(3r^2+h^2)/12$	$m \cdot r^2/2$
Esfera (r)	$2m \cdot r^2/5$	$2m \cdot r^2/5$	$2m \cdot r^2/5$

robo_hospitalar: quatro links, três joints

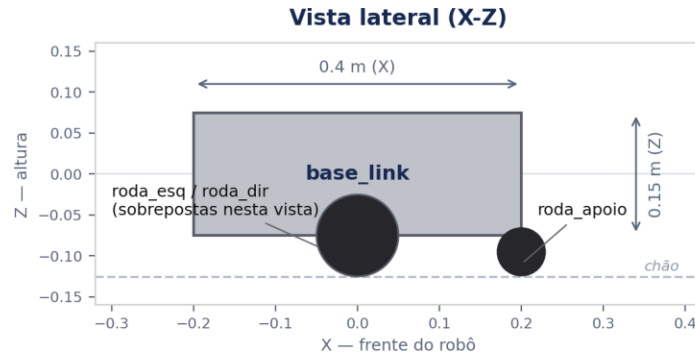
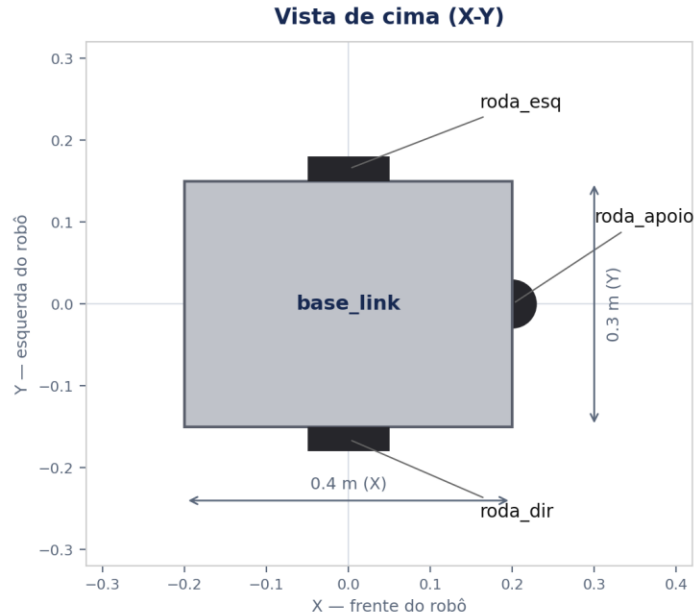
base_link no centro, três joints ligando as rodas



base_link é o parent nos três joints — roda_esq, roda_dir e roda_apoio são todos child diretos, nenhum se conecta aos outros.

Como o robô vai ficar

Vistas superior e lateral, nas dimensões exatas do URDF



robo_hospitalar.urdf — parte 1 de 6

Cabeçalho e base_link: caixa $0.4 \times 0.3 \times 0.15$ m

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<?xml version="1.0"?>
<robot name="robo_hospitalar">

  <!-- base_link: chassi 0.4x0.3x0.15 m (X x Y x Z), origem no centro -->
  <link name="base_link">
    <visual>
      <geometry><box size="0.4 0.3 0.15"/></geometry>
      <material name="cinza"><color rgba="0.6 0.6 0.6 1.0"/></material>
    </visual>
    <collision>
      <geometry><box size="0.4 0.3 0.15"/></geometry>
    </collision>
    <inertial>
      <mass value="5.0"/>
      <inertia ixx="0.046875" ixy="0" ixz="0"
        iyy="0.076042" iyz="0" izz="0.104167"/>
    </inertial>
  </link>
```

robo_hospitalar.urdf — parte 2 de 6

roda_esq: cilindro girado 90° para o eixo apontar em Y

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<!-- roda_esq: cilindro r=0.05 h=0.03m, girado 90° em X (eixo -> Y) -->
<link name="roda_esq">
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.05" length="0.03"/></geometry>
    <material name="preto"><color rgba="0.1 0.1 0.1 1.0"/></material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.05" length="0.03"/></geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <mass value="0.2"/>
    <inertia ixx="0.00014" ixy="0" ixz="0"
      iyy="0.00014" iyz="0" izz="0.00025"/>
  </inertial>
</link>
```

robo_hospitalar.urdf — parte 3 de 6

roda_dir: idêntica a roda_esq, só muda o nome

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<!-- roda_dir: mesma geometria de roda_esq -->
<link name="roda_dir">
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.05" length="0.03"/></geometry>
    <material name="preto"><color rgba="0.1 0.1 0.1 1.0"/></material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.05" length="0.03"/></geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <mass value="0.2"/>
    <inertia ixx="0.00014" ixy="0" ixz="0"
      iyy="0.00014" iyz="0" izz="0.00025"/>
  </inertial>
</link>
```

robo_hospitalar.urdf — parte 4 de 6

As duas juntas motrizes (continuous)

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<!-- juntas continuous: rodas giram livres em Y.  
      y = +-0.165 (metade da base 0.15 + metade da roda 0.015)  
      z = -0.075 (nivel da base do chassi) -->  
<joint name="junta_roda_esq" type="continuous">  
  <parent link="base_link"/>  
  <child link="roda_esq"/>  
  <origin xyz="0 0.165 -0.075" rpy="0 0 0"/>  
  <axis xyz="0 1 0"/>  
</joint>  
  
<joint name="junta_roda_dir" type="continuous">  
  <parent link="base_link"/>  
  <child link="roda_dir"/>  
  <origin xyz="0 -0.165 -0.075" rpy="0 0 0"/>  
  <axis xyz="0 1 0"/>  
</joint>
```

robo_hospitalar.urdf — parte 5 de 6

roda_apoio: esfera de 0.03m (caster)

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<!-- roda_apoio: esfera r=0.03m, funciona como caster -->
<link name="roda_apoio">
  <visual>
    <geometry><sphere radius="0.03"/></geometry>
    <material name="preto"/>
  </visual>
  <collision>
    <geometry><sphere radius="0.03"/></geometry>
  </collision>
  <inertial>
    <mass value="0.05"/>
    <inertia ixx="0.000018" ixy="0" ixz="0"
              iyy="0.000018" iyz="0" izz="0.000018"/>
  </inertial>
</link>
```

robo_hospitalar.urdf — parte 6 de 6

A junta fixed da roda de apoio, e o fechamento do arquivo

salvar em `comunicacao_ros2/urdf/robo_hospitalar.urdf`

```
<!-- junta fixed: x=0.2 (borda frontal do chassi),  
      z=-0.095 (mais baixa, compensando o raio menor  
      da esfera, para manter o chassi nivelado) -->  
<joint name="junta_roda_apoio" type="fixed">  
  <parent link="base_link"/>  
  <child link="roda_apoio"/>  
  <origin xyz="0.2 0 -0.095" rpy="0 0 0"/>  
</joint>  
  
</robot>
```

Validando a sintaxe com `check_urdf`

`check_urdf` — critério de validação do AT: 'URDF sintaticamente correto'

```
$ check_urdf \  
  ~/tp1_ws/src/comunicacao_ros2/urdf/robo_hospitalar.urdf  
  
robot name is: robo_hospitalar  
----- Successfully Parsed XML -----  
root Link: base_link has 3 child(ren)  
  child(1): roda_esq  
  child(2): roda_dir  
  child(3): roda_apoio
```

Não achou o comando?

`check_urdf` vem do pacote `liburdfdom-tools`: `'sudo apt install liburdfdom-tools'` resolve. A árvore impressa também confirma a estrutura `parent/child` do diagrama.

Visualizando no RViz2

Confirmação visual antes do Gazebo (Etapa 9) — `check_urdf` só valida sintaxe, não aparência

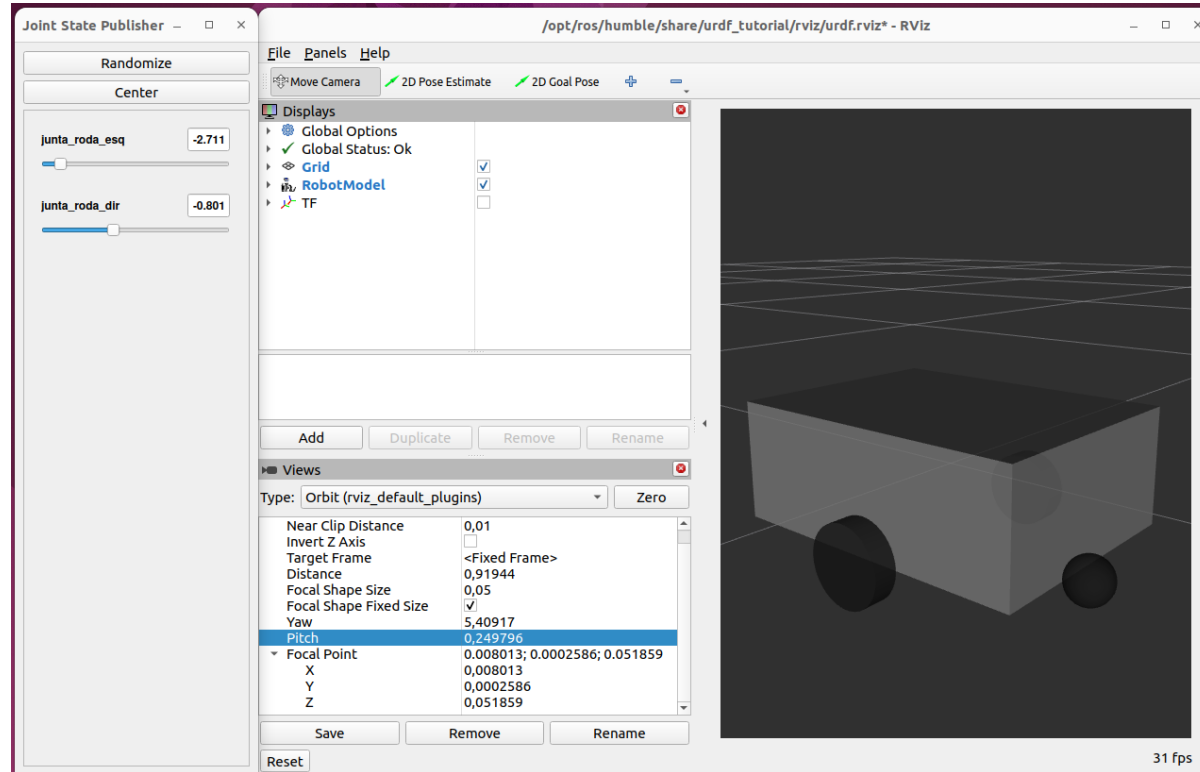
```
ros2 launch urdf_tutorial display.launch.py  
model:=$HOME/tp1_ws/src/comunicacao_ros2/urdf/robo_hospitalar.urdf
```

(abre o RViz2 + sliders para as juntas continuous)

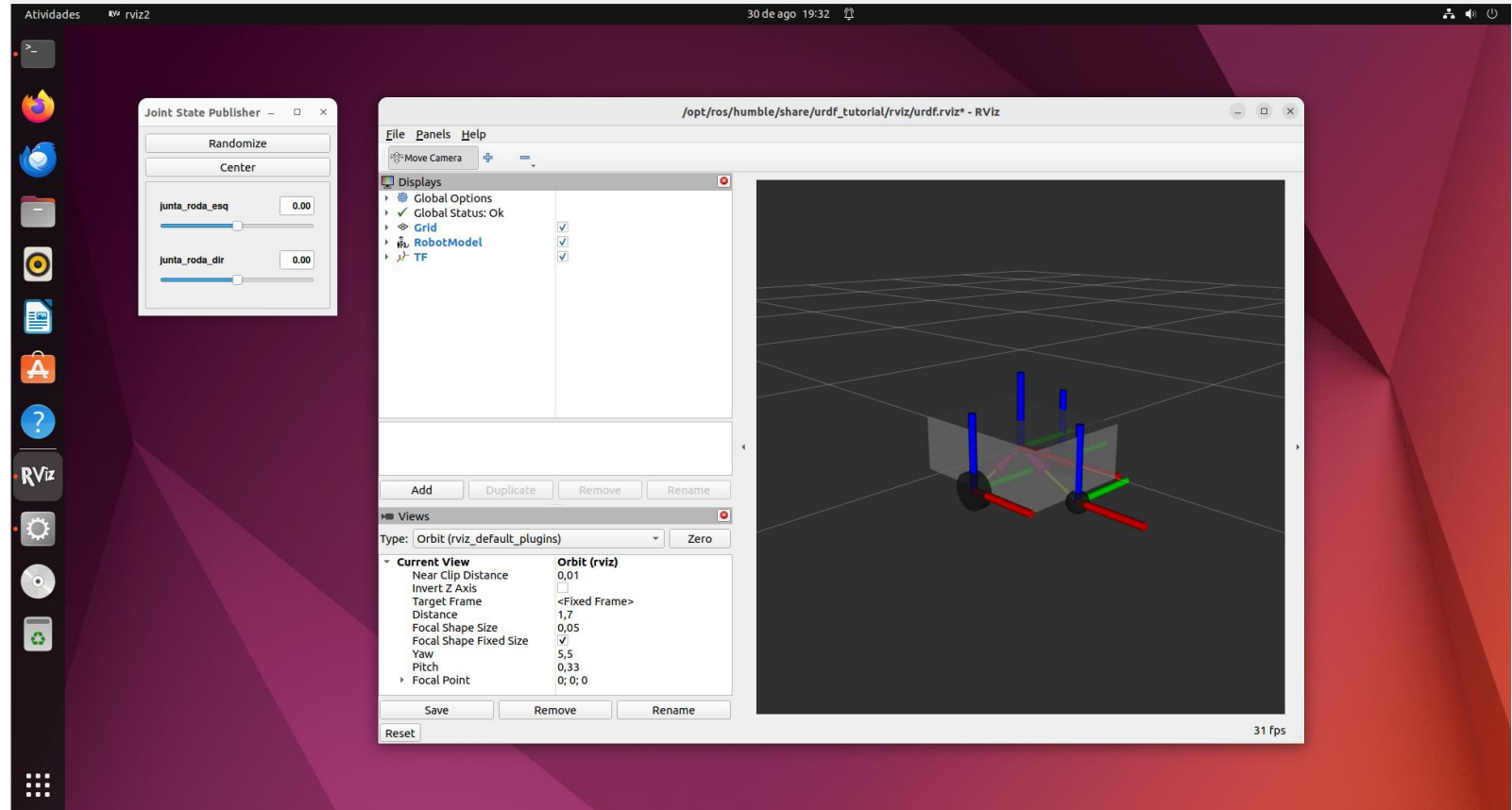
Não achou o pacote?

'sudo apt install ros-humble-urdf-tutorial ros-humble-joint-state-publisher-gui' resolve. Use os sliders de `junta_roda_esq/junta_roda_dir` para confirmar que as rodas giram.

RViz



RViz



URDF sem armadilhas comuns

- rpy é sempre em RADIANOS — se um ângulo parecer absurdo (tipo 90.0), provavelmente esqueceram de converter de graus
- collision não precisa ser idêntica à visual — formas simples (caixas, cilindros) na collision aceleram a simulação física
- Todo link (exceto às vezes o primeiro, 'root') precisa de exatamente um joint que o conecte a um parent — um link 'solto' quebra a árvore
- Nomeie links e joints de forma consistente (roda_esq / junta_roda_esq) — facilita ler o 'check_urdf' e o grafo de TF depois

robo_hospitalar: links e joints, lado a lado

Link	Geometria	Joint (tipo)	Posição relativa
base_link	box 0.4×0.3×0.15	— (root)	origem do robô
roda_esq	cylinder r=0.05 h=0.03	continuous	y=+0.165, z=-0.075
roda_dir	cylinder r=0.05 h=0.03	continuous	y=-0.165, z=-0.075
roda_apoio	sphere r=0.03	fixed	x=+0.2, z=-0.095

OK

`<link>` com
`visual/collision/inertial`

os quatro links do `robo_hospitalar.urdf` completos

OK

`<joint>` `continuous` vs `fixed`

cada tipo e como definir `<axis>`

OK

`rpy` em `radianos`

converter graus para radianos e explicar por quê

OK

`check_urdf`

URDF validado sintaticamente, árvore de 3 filhos confirmada

Próximo passo

Com `robo_hospitalar.urdf` pronto e validado, a segunda metade do AT Exercício 3 está coberta — falta só a integração final. Na Etapa 9, vamos subir esse robô dentro do Gazebo com um Launch file Python, orquestrando `robot_state_publisher`, `spawn_entity.py` e o `monitor_transporte.py` da Etapa 7, tudo de uma vez. É o fechamento do AT.